

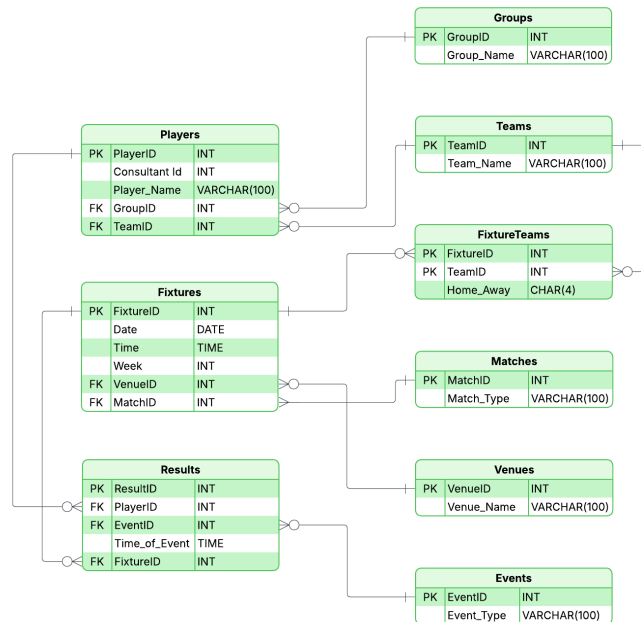


Database Design Project - Football League

Project Goal:

To build a relational database model that organises football league data in a structured and efficient manner, accurately represents relationships between entities, prevents data redundancy and inconsistency through normalisation, and supports reliable data storage while allowing the database to be effectively queried and updated.

ER Diagram:



Data Dictionary:

Entity Name	Field Name	Data Type	Constraints	Assumptions	Description	Example
Players, Results	PlayerID	INT	Primary Key, Foreign Key		Unique number ID for each player	12
Players	Consultant Id	INT			Unique number ID assigned by company	201
Players	Player_Name	VARCHAR(100)	NOT NULL	Every player has a name	First and last name of each player	Mara Barnett

Teams/ FixtureTeams, Players	TeamID	INT	Primary Key, Foreign Key		Unique number ID for each team	4
Teams	Team_Name	VARCHAR(100)	NOT NULL	Every team must have a name	Name assigned to each team	Data Cleaners
Groups, Players	GroupID	INT	Primary Key, Foreign Key		Unique number ID for each group	7
Groups	Group_Name	VARCHAR(100)	NOT NULL	Every group must have a name	Name assigned to each group	Cohort 6
Matches, Fixtures	MatchID	INT	Primary Key, Foreign Key		Unique number ID for each match type	2
Matches	Match_Type	VARCHAR(100)	NOT NULL	Every match type must have a name	Name assigned to each match type	Friendly
Venues, Fixtures	VenueID	INT	Primary Key, Foreign Key		Unique number ID for each venue	3
Venues	Venue_Name	VARCHAR(100)	NOT NULL	Every venue must have a name	Name assigned to each venue	Wimbledon 2
Fixtures/ FixtureTeams, Results	FixtureID	INT	Primary Key, Foreign Key		Unique number ID for each fixture	11
Fixtures	Date	DATE	YYYY-MM-DD	The date of a fixture may not be assigned yet	Scheduled date for each fixture	2022-10-08
Fixtures	Time	TIME	HH:MM	The time of a fixture may not be assigned yet	Scheduled time for each fixture	16:00
Fixtures	Week	INT		The week of a fixture may not be assigned yet	Assigned week for each fixture	5
FixtureTeams	Home_Away	CHAR(4)	Either "Home" or "Away"	Only 2 options. A team's home or away status may not be assigned yet	Home or Away status of each team in a given fixture	Home
Events, Results	EventID	INT	Primary Key, Foreign Key		Unique number ID for each event type	2
Events	Event_Type	VARCHAR(100)	NOT NULL	Every event type must have a name	Name assigned to each event type	Goal
Results	ResultID	INT	Primary Key		Unique number ID for each	27

					result	
Results	Time_of_Event	TIME	H:MM:SS	A fixture cannot last more than 2 hours. 90 min game + max 30 min penalty	The hour, minute, and second of fixture that an event occurred	1:22:00

Design Rationale:

Players:

- Created a players table to store player-specific attributes.
- Reassigned players with unique PlayerID's to account for duplicates in originally assigned Consultant ID's.

Teams:

- Created a teams table to store team-specific attributes.
- Created a TeamID to uniquely identify each record in the table.

Groups:

- Created a groups table to store group-specific attributes.
- Created a GroupID to uniquely identify each record in the table.

Matches:

- While no match related attributes (other than Match_Type) exist in the table, Matches table was created to enable future input of match-related attributes, dependent on the match type.
- Created a MatchID to uniquely identify each record in the table.

Venues:

- Created a venues table to store venue-specific attributes.
- Created a VenueID to uniquely identify each record in the table.

Fixtures:

- Created a fixtures table to store fixture-specific attributes.
- Created a FixtureID to uniquely identify each record in the table.
- Separated date and time attributes to allow for analysis of game results by time or date as new games, with potentially different times, are added to the database.
- Edited F12's date to match F11's date, following the rule of the other fixtures (each week has two games, played on the same day).

FixtureTeams:

- Created as a junction table to resolve the many-to-many relationship between Teams and Fixtures and store attributes dependent on both the fixture and team.
- Uses a composite primary key (FixtureID, TeamID) as each fixture involves multiple teams. The Home_Away attribute is dependent on both FixtureID and TeamID, indicating whether a specific team is playing at home or away in a given fixture.

- Home_Away data type set as CHAR(4). "Home" and "Away" are the only valid values, both consisting of four characters. Setting this data prevents invalid data input.

Events:

- Created a events table to store event-specific attributes.
- Created a EventID to uniquely identify each record in the table.

Results:

- Created a fixtures table to store result-specific attributes.
- Created a ResultID to uniquely identify each record in the table.

Database Assumptions:

Players:

- Many players can belong to a single team OR not yet been assigned to a group (zero-or-many-to-one)
- Many player can belong to a single group OR not yet been assigned to a group (zero-or-many-to-one)
- A single player can be associated with multiple results OR not have produced any results (one-to-zero-or-many)

Teams:

- A single team can consist of many players OR not yet have any players (one-to-zero-or-many)
- A single team can be referenced by multiple FixtureTeam records OR have not yet been assigned to a fixture (one-to-zero-or-many)

Groups:

- A single group can consist of many OR not yet have any players (one-to-zero-or-many)

Matches:

- A single match type can be associated with multiple fixtures (one-to-many)

Venue:

- A single venue can be associated with multiple fixtures OR not yet been assigned to a fixture (one-to-zero-or-many)

Fixtures:

- Multiple fixtures can reference a single venue OR have not yet been assigned to a venue (many-or-zero-to-one)
- Multiple fixtures can reference a single match type (many-to-one)
- A single fixture can be referenced by multiple FixtureTeam records OR have not yet been assigned any teams (one-to-zero-or-many)
- A single fixture can be associated with multiple results OR not produce any results (one-to-zero-or-many)

FixtureTeams:

- Many or no FixtureTeam records can reference a single team (many-or-zero-to-one)
- Many or no FixtureTeam records can reference a single fixture (many-or-zero-to-one)

Events:

- A single event can be associated with multiple results OR not have occurred in any results (one-to-zero-or-many)

Results:

- Many results can refer to a single player OR no results can be produced for a player (zero-or-many-to-one)
- Many results can refer to a single event OR no results can reference a particular event (zero-or-many-to-one)
- Many results can refer to a single fixture OR no results can be produced by a fixture (zero-or-many-to-one)

Future Work:**Potential new attributes for each entity:****Players:**

- Age, INT or DateOfBirth, DATE, YYYY-MM-DD
 - Calculate average age of each team
 - Ensure equal age distribution across teams
- Height, INT
 - Calculate average height of each team
 - Ensure equal height distribution across teams

Teams:

- ManagerID, INT
 - Create a new manager table detailing the manager name and other manager-specific attributes

Fixture:

- RefereeID, INT
 - Create a new referee table detailing the referee name and other referee-specific attributes
- Status, VARCHAR(100), e.g. *Scheduled, Postponed, Completed, Cancelled*
 - Filter relevant data and query accurately, e.g. excluding unplayed matches

Venue:

- Capacity, INT
 - Filter data for analysis, e.g. match type by venue
- SurfaceType, VARCHAR(100), e.g. *grass or artificial*
 - Filter data for analysis, e.g. how surface type influences fixture results

Example Query:

```
UPDATE Player
SET Age = CASE
WHEN PlayerID = 'P1' THEN 20
WHEN PlayerID = 'P2' THEN 22
```

```
WHEN PlayerID = 'P3' THEN 19
END;
```

What happens if a player changes teams?

Option 1: UPDATE

Use the SQL command UPDATE to reassign the player's team.

Example Query:

```
UPDATE Player
SET TeamID = 4
WHERE PlayerID = 1;
```

However, this will result in a loss of a player's historical team data.

Option 2: Create a new table

Introduce a new PlayerTeams table to track team changes over time.

PlayerTeams

	Field Name	Data Type
PK	PlayerID	INT
PK	TeamID	INT
	Start_Date	DATE
	End_Date	

What if a new season is added to the database?

Step 1: Create a new table

Introduce a new Season table to filter and analyse data across different time periods.

	Field Name	Data Type
PK	SeasonID	INT
	Season_Name	VARCHAR(7)*
	Start_Date	DATE
	End_Date	DATE

*e.g., "2022/23"

Step 2: Establish a relationship with the Fixtures table

Add SeasonID to the Fixtures table as a foreign key, enabling relationships between other tables through Fixtures.

Example Query:

```
SELECT *
FROM Fixture
WHERE SeasonID = 2;
```

Conclusion:

This database model effectively organises football league data, ensuring data integrity through normalisation and clearly defined relationships. It supports efficient querying and updating while remaining scalable for

future enhancements such as season tracking and additional attributes.

Reorganised Data:

Excel File: TeannaOttey_SQL_01_09_Project_DatabaseDesign_FootballLeague_Tables